

Interoperable Agentic AI with MCP

Local-First

Giuseppe Mastrandrea

PyCon Italia 2026

Chi sono 🙌

Giuseppe Mastrandrea

- 💻 Front-end developer — 2011 → 2020
- 🧑🏫 Insegnante — dal 2020
- 🎓 Formatore e docente @ Data Masters — dal 2021

Un modello vale quanto il suo contesto

"Models are only as good as the context provided to them."

Un modello da frontiera **senza accesso al mondo esterno** vale meno di un modello mediocre che può:

- leggere i tuoi file, il tuo DB, le tue issue
- scrivere su Slack, aprire una PR, creare un ticket
- farlo **conversando**, con un'integrazione a monte

La sfida oggi è **come portare il contesto al modello** in modo scalabile.

Il problema

Costruisco un agent che legge issue GitHub e apre task su Linear. Funziona.

Poi:

- arriva **GPT-5** → riscrivo il tool calling per OpenAI
- un collega lo rifà in **CrewAI**
- migriamo a **LangGraph** → da rifare ancora

$N \text{ stack AI} \times M \text{ tool} = N \times M \text{ integrazioni.}$

5 stack $\times$ 10 tool = **50 integrazioni** da scrivere e mantenere.

"I framework non bastano?"

LangChain, LlamaIndex, CrewAI hanno un'astrazione `Tool` riusabile tra LLM.
Scrivi una volta, gira su Claude, GPT, Gemini.

Limite: il tool vive *dentro* il framework.

Il tuo `GitHub tool` LangChain è inaccessibile a Claude Desktop, Cursor, una webapp custom, un altro framework.

*Framework: **N modelli $\times$ M tool.***

*MCP: **N stack $\times$ M tool** — lo stack include i framework stessi.*

La soluzione: un protocollo ✨

MCP — Model Context Protocol

Una **specifica aperta** su come modelli e tool si parlano.

- 📐 Handshake, primitive, formato messaggi → **standard**
- 🛠 Linguaggio, framework, implementazione → **liberi**
- 🤝 Adottato da Anthropic, OpenAI, Google, Microsoft

Scritto una volta. Usato ovunque.

L'analogia HTTP

Prima di HTTP

Gopher
FTP
WAIS
...
} caos

Dopo HTTP

Browser A
Browser B → HTTP → Server
Browser C

 "Tutti parliamo questa lingua."

MCP porta questo modello tra **host AI** (Claude Desktop, IDE, app, agenti) e **server di tool**.

L'eredità di LSP

MCP si ispira a **LSP — Language Server Protocol** (Microsoft, 2016).

Anno	Protocollo	Standardizza...
1991	HTTP	Browser ↔ web server
2016	LSP (Microsoft)	IDE ↔ tool di linguaggio
2024	MCP (Anthropic)	Host AI ↔ tool & data

Prima di LSP: $N \text{ IDE} \times M \text{ linguaggi} = N \times M \text{ plugin}$.

Prima di MCP: $N \text{ modelli} \times M \text{ tool} = N \times M \text{ integrazioni}$.

*Stesso problema. Stessa soluzione. **Uno standard in mezzo.***

Quando usare cosa

Approccio	Cosa fa	Limite
 Tool calling	LLM + tool dentro l'app	Tool non riusabili
 Agent framework (LangChain, CrewAI)	Orchestrazione multi - step	Tool accoppiati al framework
 MCP	Tool / resource / prompt portabili tra client, LLM, framework	—

 *MCP **affianca** gli agenti: rende **portabili** i tool, le resource e i prompt che usano. Si usa **con** LangChain, **con** CrewAI, **con** Claude Desktop.*

"MCP è morto"?

Le critiche da prendere sul serio:

-  **Context bloat** — i server MCP bruciano token già al boot
-  **Auth doppia** — login nel sistema **e** nel server MCP
-  **Output verboso** — JSON-RPC pesa più del testo nativo
-  **Se esiste già un CLI, MCP è overhead** — persino Claude Code preferisce Bash a 30 server MCP

"MCP is dead. The CLI is winning the AI agent stack."

— HackerNoon, 2025

 ...ma chi scrive così è **un developer che parla ad altri developer.**

Per chi non programma

Il developer è lo **0.1% dell'utenza AI**.

Il restante 99% non scriverà **mai** una riga di codice.

Utente reale	Cosa sa fare	Cosa NON sa fare
 Product manager	Aprire Claude Desktop	Scrivere bash
 Docente	Chattare con ChatGPT	Orchestrare un agente
 Medico	Usare un'app	Costruire con LangChain

Per loro la scelta è secca: **MCP o nessuna capability esterna**.

La CLI è una lingua che non parlano.

Il modello browser-plugin

Il parallelo giusto arriva dal **web**:

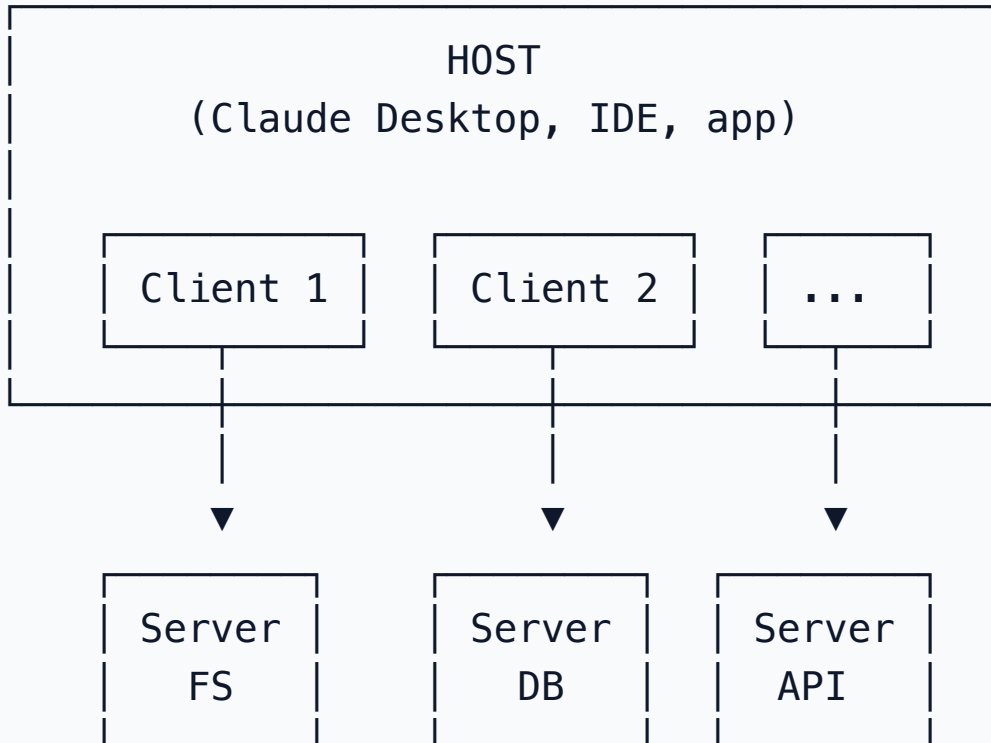
|  ***Browser : estensione = Host AI : server MCP***

L'utente **installa** un server già pronto.

Qualcun altro l'ha scritto **una volta sola**, per tutti.

 Per il 99% degli utenti AI, MCP è **l'unica strada** verso capability esterne.

L'architettura MCP



1 Host → N Client → 1 Server per Client (1:1)

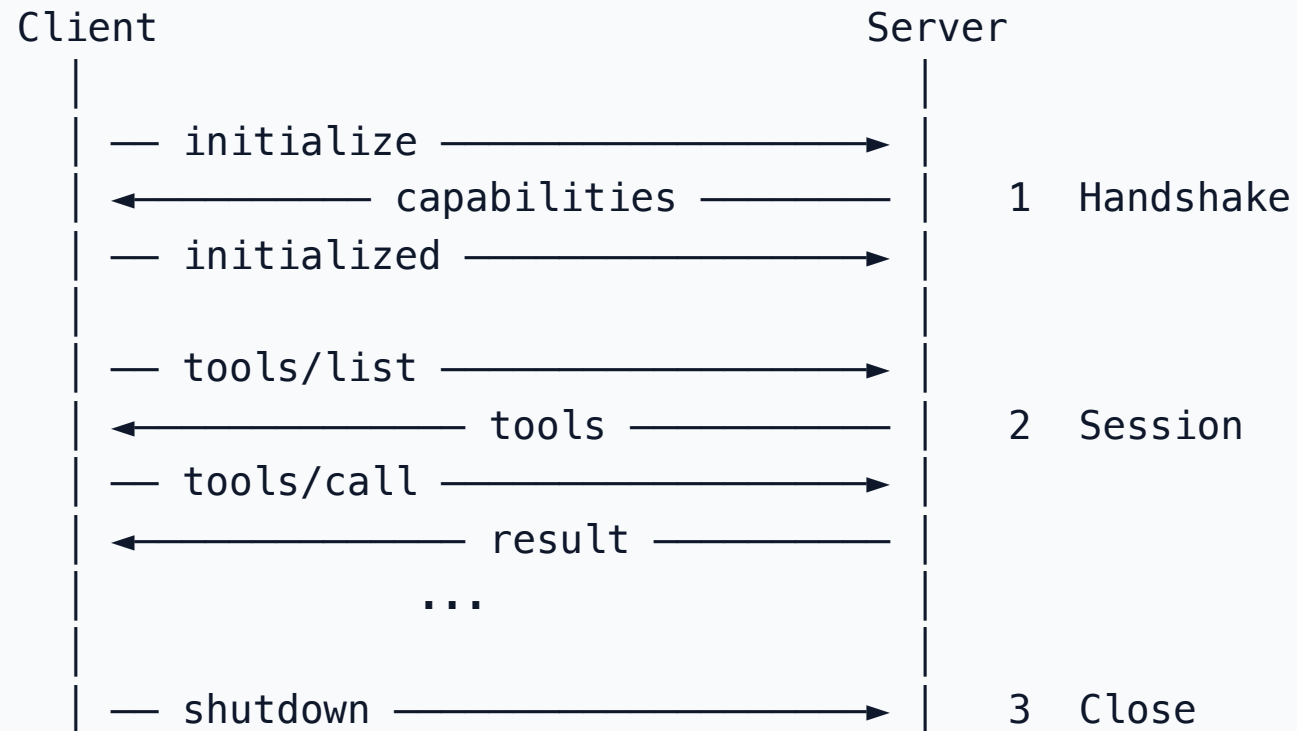
Il protocollo sotto il cofano

JSON-RPC 2.0 — semplice, testuale, stateful.

```
// → request
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list"
}
```

```
// ← response
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "get_weather",
        "description": "Current weather",
        "inputSchema": {
          "type": "object",
          "properties": {
            "city": {"type": "string"}
          },
          "required": ["city"]
        }
      }
    ]
  }
}
```

II lifecycle



Initialize → negotiate → session → shutdown

Comunicazione bidirezionale

I messaggi viaggiano **in entrambe le direzioni**.

```
graph LR; C[Client] -- request --> S[Server]; S -- response --> C; S -- notification --> C; S -- "sampling (LLM)" --> C;
```

Client ← request → Server
Client ← response → Server
Client ← notification → Server
Client ← sampling (LLM) → Server

Sampling — il server può chiedere al client di chiamare l'LLM per suo conto:

- l'utente resta padrone della chiave API e del modello
- il server non paga per i token
- si possono scrivere server "intelligenti" indipendenti dal provider

Anche le **notifiche** sono bidirezionali: il server può avvisare il client che la lista dei tool è cambiata, che una risorsa si è aggiornata, ecc.

Le tre primitive

Primitiva	Controllata da	Analogia HTTP
 Tool	Model - controlled	POST — azione con side-effect
 Resource	Application - controlled	GET — dato letto nel contesto
 Prompt	User - controlled	Template / form

*Chi decide **quando** invocare la primitiva cambia tutto.*

Tool — Model-controlled

Il **modello** decide quando chiamare il tool e con quali parametri.

```
@mcp.tool()
def get_weather(city: str, units: str = "metric") -> dict:
    """Restituisce il meteo corrente per una città.

    Args:
        city: nome della città (es. "Roma")
        units: "metric" o "imperial"
    """
    return weather_api.fetch(city, units)
```

 Il **docstring** è il contratto con l'LLM.

Nome, descrizione e schema guidano la scelta del modello.

Resource — Application-controlled

Read-only. È l'host che decide cosa caricare nel contesto.

```
@mcp.resource("file:///docs/{name}")
def get_doc(name: str) -> str:
    """Documenti di progetto."""
    return (DOCS_DIR / name).read_text()
```

- 🔍 Semantica **GET** — pura lettura
- 📁 Dati statici o derivati: file, righe DB, log, config
- 🚫 Nessun side-effect

Prompt — User-controlled

Template che generano **messaggi pronti** per l'LLM.

```
@mcp.prompt()
def review_pr(diff: str, focus: str = "security") -> str:
    """Slash command: /review-pr"""
    return f"Rivedi questo diff con focus su {focus}:\n\n{diff}"
```

-  L'**utente** sceglie quando attivarli
-  Esposti come slash command o azioni rapide
-  Building block riusabili per workflow ricorrenti

"Ma allora è solo un'API?"

La domanda che arriva sempre. Risposta breve: **quasi**.

	REST API	MCP Server
Chi parla	Codice ↔ codice	LLM ↔ tool
Contratto	OpenAPI / Swagger	inputSchema JSON + docstring
Discovery	Documentazione PDF	tools/list runtime
Auth	Spesso embedded nel client	Negoziata via protocollo
Riusabilità	Per <i>quel</i> client	Per <i>qualsiasi</i> host MCP

Un MCP server è spesso un **wrapper in linguaggio naturale** sopra un'API esistente: la stessa API REST resta lì, ma diventa **parlabile** da un modello.

I trasporti 🚚

Trasporto	Quando	Setup
 stdio	Locale, in-process	<code>mcp.run()</code>
 streamable HTTP	Remoto, distribuito	<code>mcp.run(transport="http")</code>

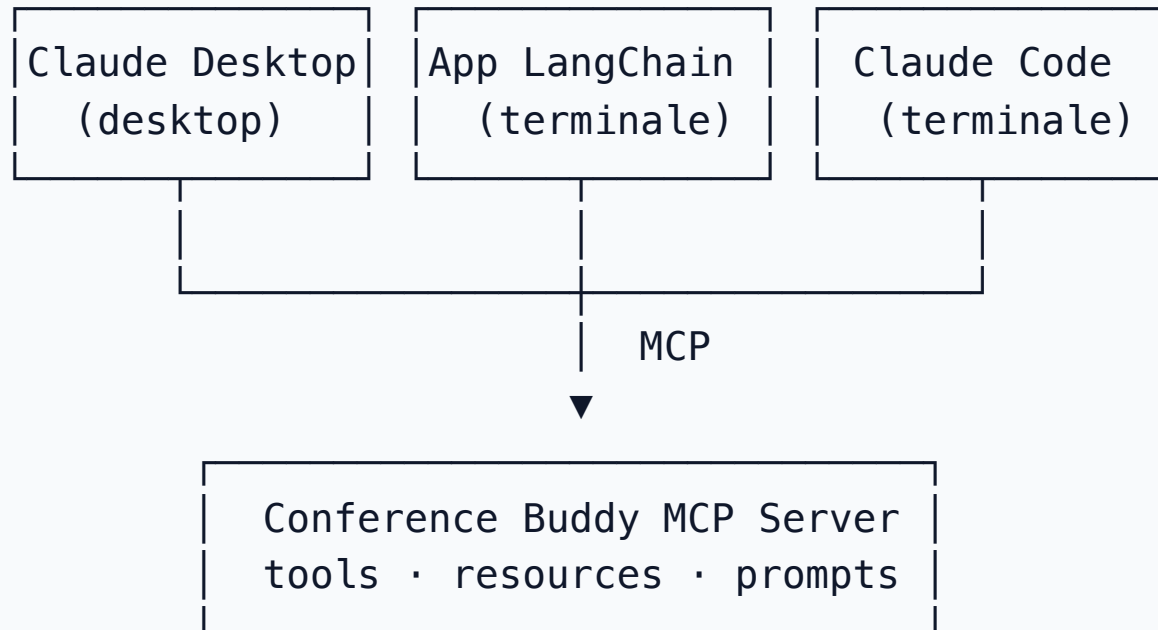
```
if __name__ == "__main__":  
    mcp.run(transport="stdio")    # locale  
    # mcp.run(transport="http")  # remoto
```

✨ Una riga di codice cambia il trasporto.

Un po' di storia — la prima spec usava **HTTP + Server-Sent Events**: stateful, ottimo per push dal server, scomodo da scalare (ogni client = una connessione viva). **Streamable HTTP** (2025) unifica i due mondi: POST stateless di default, GET → SSE *solo se serve* lo streaming.

Cosa costruiremo oggi 🛠️

Conference Buddy 🎤



Roadmap → ① tool → ② mono-server → ③ multi-server → ④ HTTP 🚀

Bonus per i partecipanti al workshop

Sconto 50% sul Percorso AI Developer di Data Masters: riservato ai partecipanti di questo workshop.

Inquadra il QR code qui sotto o vai su datamasters.it/catalogo/percorsi-di-carriera/ai-developer per sfruttare il codice promo.

